

1 JAVA REST Client: Instruction

1.1 Purpose

Use the following instruction if you want to call a web service on a third-party system via Web Service Provider (WSP) from a JAVA program part using http or https.

Here, the WSP acts as client.

1.2 Requirements

The required libraries are available as of service pack 16.



You must ensure that the following files are included in the class path:

MpdvCommonWebUtil.jar, *MpdvDomCoreSdiCompileLib.jar* and *MpdvDomCoreUserExitCompileLib.jar*.



This tutorial uses help functions from the library *MpdvCommonWebUtil.jar*. You need not use this library to call REST services. But it is much easier to call REST services with this library than with JAVA.



To create a trust store, you require OpenSSL (<https://www.openssl.org/>) and the JAVA tool "keytool" from a JAVA JDK. For basic tasks, these two programs are not required. They are only required for the extension "version with use of a trust store".

1.3 Task

Use the MES Development Suite to call the REST service "https://api.predic8.de/shop/products/" from the user exit sdiGlobalModifyRequest of the service MDUnit.list. This user exit stores the JSON result of the REST call as file in a system path. The user exit is called before the actual service processing is started.

1.4 Activate the development mode

Edit the file "<InstallDir>\jdir\MOC\1\config.properties" and insert the line "development.mode = 1" below "configreload.timeout=...".

```
#####
#####INSTANCE CONFIGURATION (1)#####
#####
# Please make sure that after configuration values is no TAB or SPACE

# Config reload timeout in seconds
# After this timeout it is checked, if the instance configuration (this file) has changed and the config must be reloaded
configreload.timeout=180

development.mode=1
```

Restart the WSP after the configuration file was changed.

Among other things, the option "development.mode = 1" ensures that changes to user exits take effect the next time the relevant service is called, without you having to restart the WSP each time.

1.5 Create system paths

The tutorial requires 2 system paths. The first path is used to store the JSON result of the REST call. The second path is used to store a JAVA trust store. The JAVA trust store includes the server certificate of the URL called.

1. Create the following folders on the server.
 - a. <InstallDir>\<SystemNo>\custom\rest_out
 - b. <InstallDir>\<SystemNo>\custom\truststore
1. Start the application *Paths* on the client.
2. Create a new system path "RESTOUT".
 - a. Protocol: "file"
 - b. Host: "localfile"
 - c. Port: 0
 - d. URL path: "<InstallDir>\<SystemNo>\custom\rest_out"
 - e. Description: "Storage for REST JSON files"
3. Create a new system path "TRUSTSTR".
 - a. Protocol: "file"
 - b. Host: "localfile"
 - c. Port: 0
 - d. URL path: "<InstallDir>\<SystemNo>\custom\truststore"
 - e. Description: "Storage for REST trust store"

1.6 Create user exit class

Create a class "MdunitsList" in the package "mdunits". Always bear in mind that the names are case sensitive. In the class "MdunitsList", create the method "sdiGlobalModifyRequest".

The result is the following:

```
package mdunits;

import de.mpdv.customization.userExit.IUserExitParam;

public class MdunitsList
{
    public void sdiGlobalModifyRequest(IUserExitParam ueParam)
    {
    }
}
```

1.7 Simple version

In the simple version, the result of an external REST service is output to Stdout.

Workflow:

- When the service "MDUnits.list" is called, the system calls the user exit "sdiGlobalModifyRequest".
- In the user exit "sdiGlobalModifyRequest", we call the REST service with the URL <https://api.predic8.de/shop/products/> using the http method GET.
- The result of the REST service is output to Stdout.

1.7.1 Extend user exit by REST call

Extend the user exit so that the following code is generated:

```
package mdunits;

import de.mpdv.commonwebutil.CommonHttpClient;
import de.mpdv.customization.userExit.IUserExitParam;
import de.mpdv.sdi.data.SesException;

import java.io.IOException;

public class MdunitsList
{
    public void sdiGlobalModifyRequest(final IUserExitParam ueParam)
    {
        try
        {
            final String result = new CommonHttpClient()
                .withRequestSettings(urlcon-> urlcon.setRequestMethod("GET"))
                .communicate("https://api.predic8.de/shop/products/", null);
            System.out.println("REST output: " + result);
        } catch (final IOException exc)
        {
            final String msg = "IO-Error executing REST service";
            throw new SesException("lkInvalidState", msg, exc, msg);
        }
    }
}
```

This code uses the class CommonHttpClient from the library MpdvCommonWebUtil.jar to call the URL <https://api.predic8.de/shop/products/> using GET. The JSON result is then output by system.out.println() to Stdout. All outputs to Stdout are stored in the file <WSP directory>\log\bootlog.txt.

If an error occurs, when the REST service is called, an error message is displayed on the client with the text: "IO-Error executing REST service".



You can only use System.out.println() during development. You must replace it by Logging or remove it at the latest before productive use.

1.7.2 Compile user exit

Compile the class MdunitsList. Be careful to use the correct class path (see section 1.2 "Requirements" above).

1.7.3 Test by calling the service MDUnits.list

1. Copy the compiled class including package on the server to
<InstallDir>\jdir\MOC\1\userexit\custom. If the custom directory does not exist, create this directory.
2. Check if the following file is available after the copy process:
<InstallDir>\jdir\MOC\1\userexit\custom\mdunits\MdunitsList.class
3. Start the application *Units* on the client.
4. Request data.
5. In the log file <InstallDir>\mip<n>\<WSP directory>\log\bootlog.txt (e.g. d:\mip1\WSP1\log\bootlog.txt), you will now find the response of the REST service.

1.8 Version with storage of REST service response in JSON file

This version is based on the preceding version. The user exit additionally stores the JSON response of the REST service as file in a system path. The file gets a unique name with the current time stamp and a 5-digit random number.

1.8.1 Extend the user exit by file storage

```
package mdunits;

import de.mpdv.commonwebutil.CommonHttpClient;
import de.mpdv.customization.userexit.IUserExitParam;
import de.mpdv.sdi.data.SesException;
import de.mpdv.sdi.data.ue.GlobalUeContext;
import de.mpdv.sdi.systemutility.IPathManagementOpenFileForWritingAction;
import de.mpdv.sdi.systemutility.ISystemUtilFactory;

import java.io.IOException;
import java.io.OutputStream;
import java.nio.charset.StandardCharsets;
import java.util.Arrays;
import java.util.Calendar;
import java.util.concurrent.ThreadLocalRandom;

public class MdunitsList
{
    public void sdiGlobalModifyRequest(final IUserExitParam ueParam)
    {
        final ISystemUtilFactory factory = ueParam.get("factory");
        final GlobalUeContext context = ueParam.get("context");
        try
        {
            final String result = new CommonHttpClient()
                .withRequestSettings(urlcon-> urlcon.setRequestMethod("GET"))
                .communicate("https://api.predic8.de/shop/products/", null);
            final String fileName = createFileName(context.getHydraNow());
            writeFile(factory, result, fileName);
        } catch (final IOException exc)
        {
            final String msg = "IO-Error executing REST service";
            throw new SesException("lkInvalidState", msg, exc, msg);
        }
    }

    private static void writeFile(final ISystemUtilFactory factory,
        final String json,
        final String fileName)
    {
        final IPathManagementOpenFileForWritingAction writeAction = factory.fetchUtil("PathManagementOpenFileForWritingAction");
        try (final OutputStream out = writeAction.writeFile("RESTOUT", fileName))
        {
            final byte[] data = json.getBytes(StandardCharsets.UTF_8);
            out.write(data);
            out.flush();
        } catch (final RuntimeException | IOException exc)
        {
            final String msg = "IO-Error persisting JSON";
            throw new SesException("lkInvalidState", msg, exc, msg);
        }
    }
}
```

```

    }

    private static String createFileName(final Calendar now)
    {
        return toTimestampString(now) + fetchFiveDigitRandomNumber() + ".json";
    }

    private static String toTimestampString(final Calendar cal)
    {
        if (cal == null)
        {
            return "null";
        }
        return pad(4, Integer.toString(cal.get(Calendar.YEAR))) +
            pad(2, Integer.toString(cal.get(Calendar.MONTH) + 1)) +
            pad(2, Integer.toString(cal.get(Calendar.DAY_OF_MONTH))) +
            pad(2, Integer.toString(cal.get(Calendar.HOUR_OF_DAY))) +
            pad(2, Integer.toString(cal.get(Calendar.MINUTE))) +
            pad(2, Integer.toString(cal.get(Calendar.SECOND))) +
            pad(3, Integer.toString(cal.get(Calendar.MILLISECOND)));
    }

    private static String fetchFiveDigitRandomNumber()
    {
        return pad(5, String.valueOf(ThreadLocalRandom.current().nextInt(100000)));
    }

    private static String pad(final int length,
        final String str)
    {
        if (str.length() == length)
        {
            return str;
        }
        final char[] paddedChars = new char[length];
        Arrays.fill(paddedChars, '0');
        final char[] strChars = str.toCharArray();
        System.arraycopy(strChars, 0, paddedChars, length - str.length(), strChars.length);
        return new String(paddedChars);
    }
}

```

The method `createFileName()` creates a file name based on a time stamp. The file name then consists of a time stamp, a 5-digit random number and the extension ".json".

The method `writeFile()` uses the SDI function "PathManagementOpenFileForWritingAction" to save the JSON using the system path "RESTOUT". UTF-8 is used as code page for the JSON file.

1.8.2 Compile user exit

Compile the class `MdunitsList`. Be careful to use the correct class path (see section 1.2 "Requirements" above).

1.8.3 Test by calling the service MDUnits.list

- Copy the compiled class including package on the server to
 <InstallDir>\jdir\MOC\1\userexit\custom. If the custom directory does not exist, create this directory.
- Check if the following file is available after the copy process:
 <InstallDir>\jdir\MOC\1\userexit\custom\mdunits\MdunitsList.class
- Start the application *Units* on the client.
- Request data.
- The folder <InstallDir>\<SystemNo>\custom\rest_out on the server now includes a *.json file, which includes the content of the REST service response.
- Open the *.json file using any text editor.

1.9 Version with use of trust store

The preceding versions accepted any server without checking the server certificate. You will extend the user exit in this version so that the concrete certificate is checked. This requires a trust store. A trust store is a storage location for trusted server certificates. With the help of a trust store, a client allows the connection to trusted servers only.

1.9.1 Create the server certificate in PEM encoding



This step is not necessary if you already have a PEM encoded server certificate.

1.9.1.1 Side note: Encoding of certificates

Certificates are mainly encoded in two ways:

- PEM encoding = Base-64 encoded X.509
 - o X.509v3
 - o Text file
 - o Can be identified via prefix in text file: -----BEGIN CERTIFICATE-----
 - o Frequent extensions
 - .cer
 - .crt
 - .pem
- DER encoding
 - o Binary file
 - o Frequent extension
 - .der



Do not rely on the file extension!

Open the certificate using any text editor and check if the file includes "BEGIN CERTIFICATE". If yes, then it is a PEM encoded certificate, otherwise not.

1.9.1.2 Option 1: Export of server certificate from web browser

1. Use a browser and call the URL <https://api.predic8.de/shop/products/>.
2. Save/export the server certificate including the complete certificate chain.
 - a. This step is browser-specific. Search the internet to find out how you can export server certificates in your browser.

- b. Important: Save the certificate as PEM encoding (also called Base-64 encoded X.509). Use the file name "server.cer".

1.9.1.3 Option 2: Convert from PFX file (PKCS #12)

You require the program keytool from a Java JDK for this step (see section 1.2 "Requirements" above).

1. Identify the alias.
 - a. "<JDK-InstallDir>\bin\keytool.exe" -list -keystore <Keyfile> -storepass <password for keyfile> -v
 - b. Copy the alias or note down the alias of the concrete certificate.
2. Export the certificate.
 - a. "<JDK-InstallDir>\bin\keytool.exe" -export -alias <alias from list command> -file server.cer -keystore server.pfx -storepass <password of keyfile>
3. The result is a PEM encoded file "server.cer".

1.9.1.4 Option 3: Conversion from DER encoded certificate

You require the program openssl from OpenSSL for this step (see section 1.2 "Requirements" above).

It is assumed that the DER encoded certificate file has the file name server.crt.

1. Check whether the file is really DER encoded by opening the file with a text editor. Search for "BEGIN CERTIFICATE".
 - a. If you do not find anything, go on with step 2.
 - b. If you find "BEGIN CERTIFICATE", rename the file in server.cer and go on with section 1.9.2 "Create the trust store".
2. Convert the certificate.
 - a. <OpenSSL-InstallDir>\openssl x509 -inform DER -in server.crt -out server.cer -outform PEM
3. The result is a PEM encoded file "server.cer".

1.9.2 Create the trust store

You require the program keytool from a Java JDK for this step (see section 1.2 "Requirements" above).

In this step, you create a trust store with the file name "truststore.jks" and the password "secret".

1. Create the trust store.
 - a. "<JDK-InstallDir>\bin\keytool.exe" -import -v -trustcacerts -alias my_alias -file server.cer -keystore truststore.jks -storepass secret -noprompt
2. The result is a trust store file "truststore.jks" with password "secret". The certificates are stored under the alias "my_alias" in the trust store.

1.9.3 Extend user exit by trust store

```
package mdunits;

import de.mpdv.commonwebutil.CommonHttpClient;
import de.mpdv.commonwebutil.ICheckedSupplier;
import de.mpdv.customization.userExit.IUserExitParam;
import de.mpdv.sdi.data.SesException;
import de.mpdv.sdi.data.ue.GlobalUeContext;
import de.mpdv.sdi.systemutility.IHydraPathReadFileAction;
import de.mpdv.sdi.systemutility.IPathManagementOpenFileForWritingAction;
import de.mpdv.sdi.systemutility.ISystemUtilFactory;

import java.io.IOException;
import java.io.InputStream;
import java.io.OutputStream;
import java.nio.charset.StandardCharsets;
import java.util.Arrays;
import java.util.Calendar;
import java.util.concurrent.ThreadLocalRandom;

public class MdunitsList
{
    public void sdiGlobalModifyRequest(final IUserExitParam ueParam)
    {
        final ISystemUtilFactory factory = ueParam.get("factory");
        final GlobalUeContext context = ueParam.get("context");
        try
        {
            final String result = new CommonHttpClient()
                .withRequestSettings(urlcon -> urlcon.setRequestMethod("GET"))
                .withTruststoreFileStreamProvider(fetchTruststore(factory), "secret")
                .withCheckHostNames()
                .communicate("https://api.predic8.de/shop/products/", null);
            final String fileName = createFileName(context.getHydraNow());
            writeFile(factory, result, fileName);
        } catch (final IOException exc)
        {
            final String msg = "IO-Error executing REST service";
            throw new SesException("lkInvalidState", msg, exc, msg);
        }
    }

    private static ICheckedSupplier<InputStream, IOException> fetchTruststore(final ISystemUtilFactory factory)
    {
        return () -> factory.<IHydraPathReadFileAction>fetchUtil("HydraPathReadFileAction").openFile("TRUSTSTR",
"truststore.jks");
    }

    private static void writeFile(final ISystemUtilFactory factory,
        final String json,
        final String fileName)
    {
        final IPathManagementOpenFileForWritingAction writeAction = factory.fetchUtil("PathManagementOpenFileForWritingAction");
        try (final OutputStream out = writeAction.writeFile("RESTOUT", fileName))
        {
            final byte[] data = json.getBytes(StandardCharsets.UTF_8);
            out.write(data);
            out.flush();
        } catch (final RuntimeException | IOException exc)
        {
            final String msg = "IO-Error persisting JSON";
            throw new SesException("lkInvalidState", msg, exc, msg);
        }
    }

    private static String createFileName(final Calendar now)
    {
        return toTimestampString(now) + "_" + fetchFiveDigitRandomNumber() + ".json";
    }

    private static String toTimestampString(final Calendar cal)
    {
        if (cal == null)
        {
            return "null";
        }
        return pad(4, Integer.toString(cal.get(Calendar.YEAR))) +
            pad(2, Integer.toString(cal.get(Calendar.MONTH) + 1)) +
            pad(2, Integer.toString(cal.get(Calendar.DAY_OF_MONTH))) +
            pad(2, Integer.toString(cal.get(Calendar.HOUR_OF_DAY))) +
            pad(2, Integer.toString(cal.get(Calendar.MINUTE))) +
            pad(2, Integer.toString(cal.get(Calendar.SECOND))) +
            pad(3, Integer.toString(cal.get(Calendar.MILLISECOND)));
    }

    private static String fetchFiveDigitRandomNumber()
    {
        return pad(5, String.valueOf(ThreadLocalRandom.current().nextInt(100000)));
    }

    private static String pad(final int length,
        final String str)
    {
        if (str.length() == length)
        {
            return str;
        }
    }
}
```



```
}  
final char[] paddedChars = new char[length];  
Arrays.fill(paddedChars, '0');  
final char[] strChars = str.toCharArray();  
System.arraycopy(strChars, 0, paddedChars, length - str.length(), strChars.length);  
return new String(paddedChars);  
}  
}
```

The method `withTruststoreFileStreamProvider()` configures the trust store and activates the check of the server certificates. The method `withCheckHostNames()` ensures that the server name matches the server name included in the certificate. Using the function `fetchTruststore()`, the trust store "truststore.jks" is loaded from the system path "TRUSTSTR".

1.9.4 Compile user exit

Compile the class `MdunitsList`. Be careful to use the correct class path (see section 1.2 "Requirements" above).

1.9.5 Test by calling the service `MDUnits.list`

1. Copy the trust store "truststore.jks" on the server to `<InstallDir>\<SystemNo>\custom\truststore`
2. Check if the following file is available after the copy process:
`<InstallDir>\<SystemNo>\custom\truststore\truststore.jks`
3. Copy the compiled class including package on the server to
`<InstallDir>\jdir\MOC\1\userexit\custom`. If the custom directory does not exist, create this directory.
4. Check if the following file is available after the copy process:
`<InstallDir>\jdir\MOC\1\userexit\custom\mdunits\MdunitsList.class`
5. Start the application *Units* on the client.
6. Request data.
7. The folder `<InstallDir>\<SystemNo>\custom\rest_out` on the server now includes a *.json file, which includes the content of the REST service response.
8. Open the *.json file using any text editor.